

Rethinking vLLM on Non-CUDA GPUs

Lessons from Building vLLM-MUSA

Xiaodong Ye & Zichen Wang
— Moore Threads



Content

目录

01 Open-Source on MUSA: Support Landscape and Critical Challenges

02 Our Journey & High-Level Architecture

03 Implementation Highlights

04 Perf, Ops, Takeaways



Part

Open-Source on MUSA: Support Landscape and Critical Challenges



Supported Open-Source Model Serving Projects on MUSA



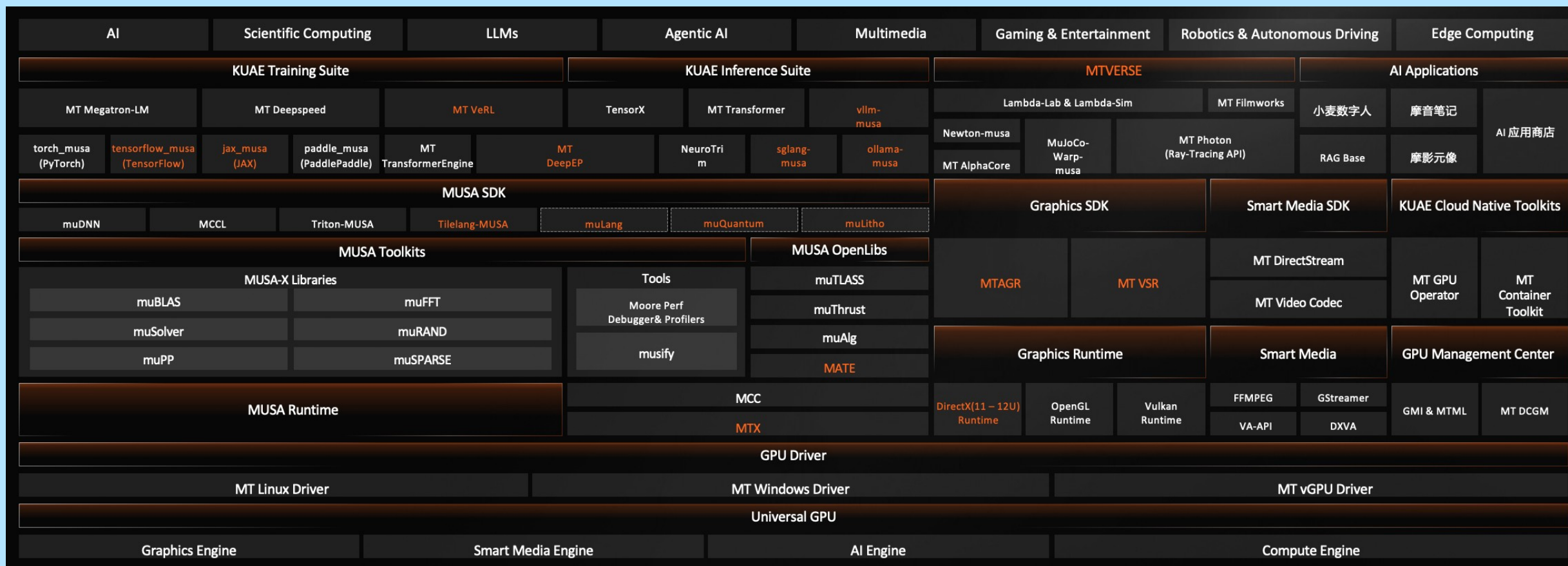
Inference Engine



Inference Deploy



MUSA Software Stack



Beyond porting: Why "run" is no longer enough



- Historically "porting" = make code run on new GPU (5 files)
- Production expectations now: maintainability, upstreamability, native performance



Bart Journey & High-Level Architecture

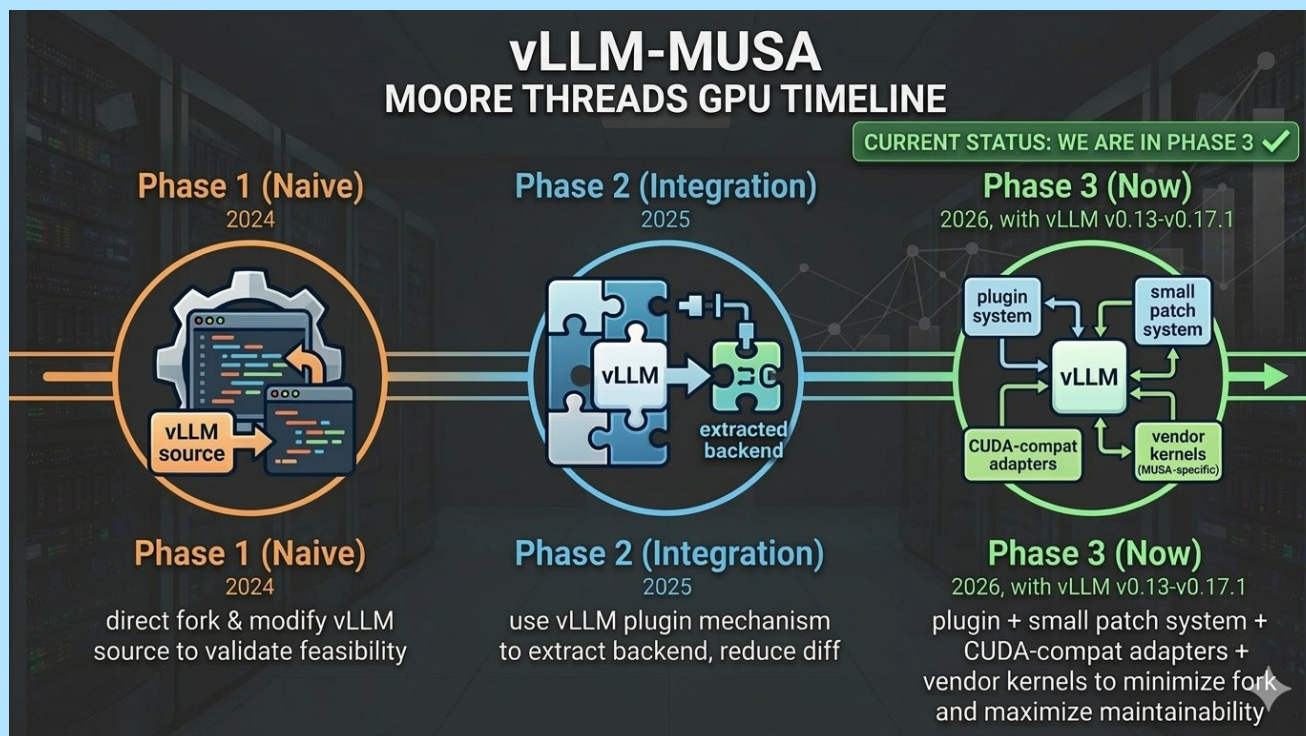


Layers to rebuild

- MUSA kernels – attention (MLA/GQA) / fused ops / MoE kernels
- CustomOp – MUSA kernels composition to achieve best performance
- Runtime Semantics – streams, graphs, async execution

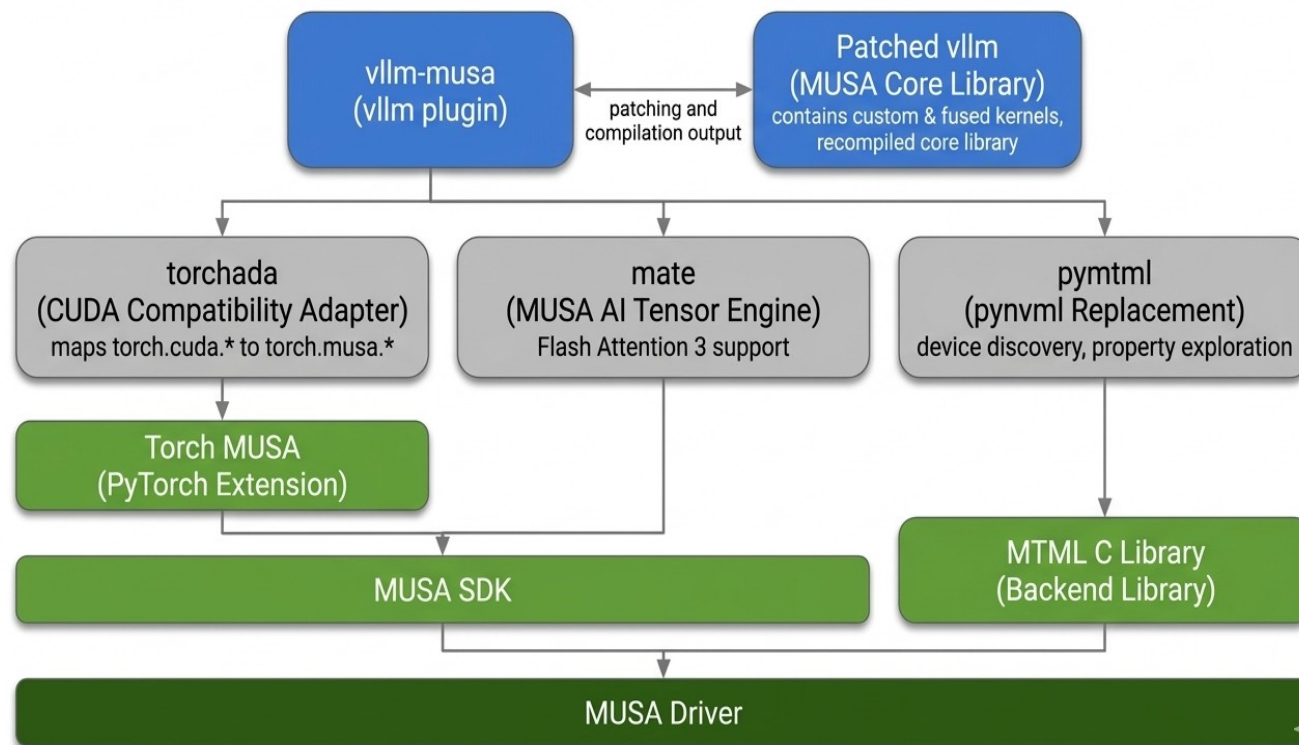


Our journey



- **Phase 1 (Naive)** — direct fork & modify vLLM source to validate feasibility.
- **Phase 2 (Integration)** — use vLLM plugin mechanism to extract backend, reduce diff.
- **Phase 3 (Now)** — plugin + small patch system + CUDA-compat adapters + vendor kernels to minimize fork and maximize maintainability.

vLLM-MUSA Overview

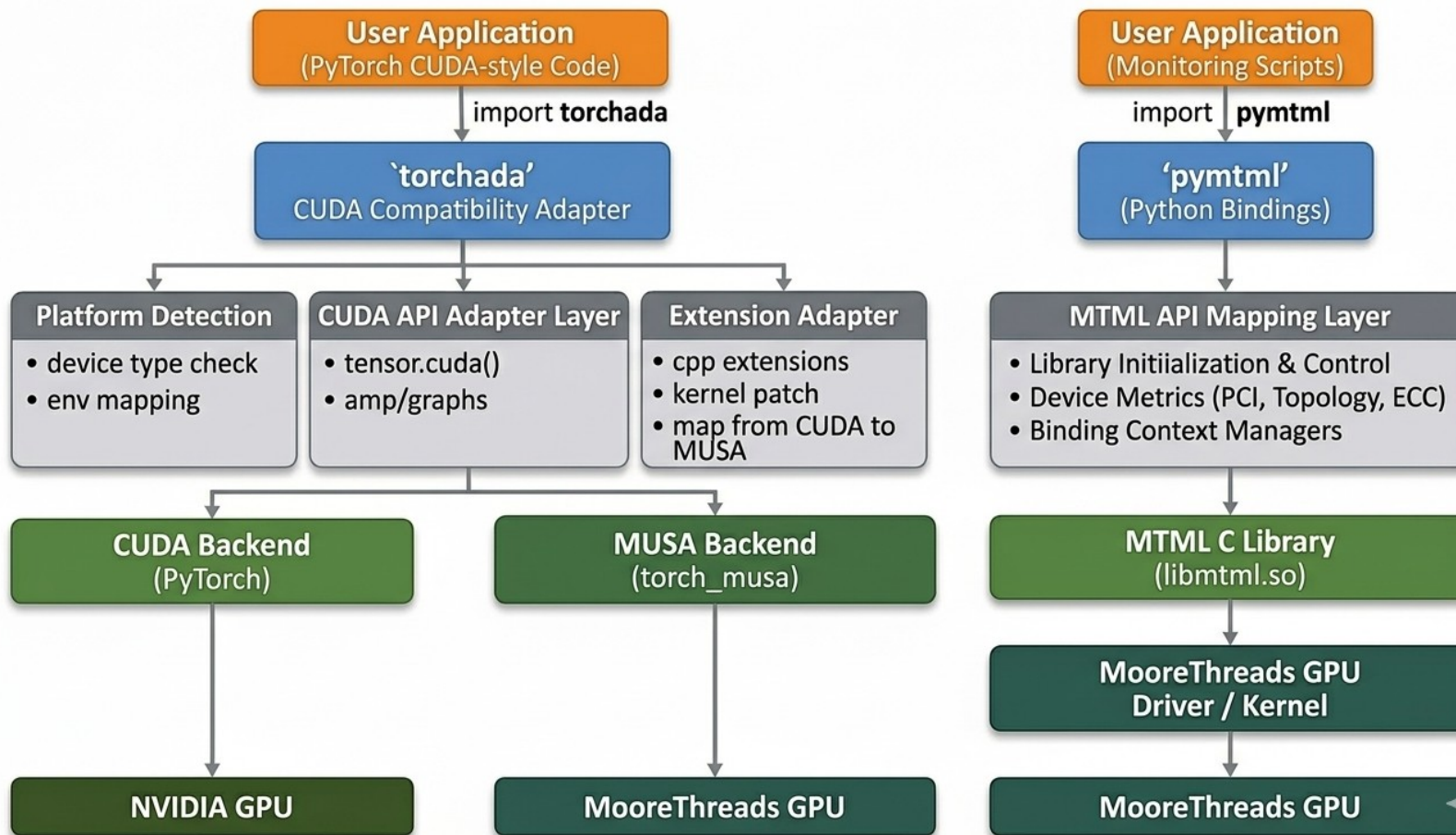


- vLLM Core (scheduler, KV cache) — kept unchanged
- Platform Plugin (OOT) — backend registration & runtime hooks
- CUDA-Compat Adapter (**torchada**) — maps `torch.cuda` semantics to MUSA
- Operator Layer (CustomOp / OOT ops) — op override, torch + kernel composition, dispatch & fallback
- Native Backend (`torch_musa` + vendor kernels) — **MATE** (FA3+FlashInfer APIs) for SOTA performance

<https://github.com/MooreThreads/vllm-musa/tree/v0.17.0-dev>



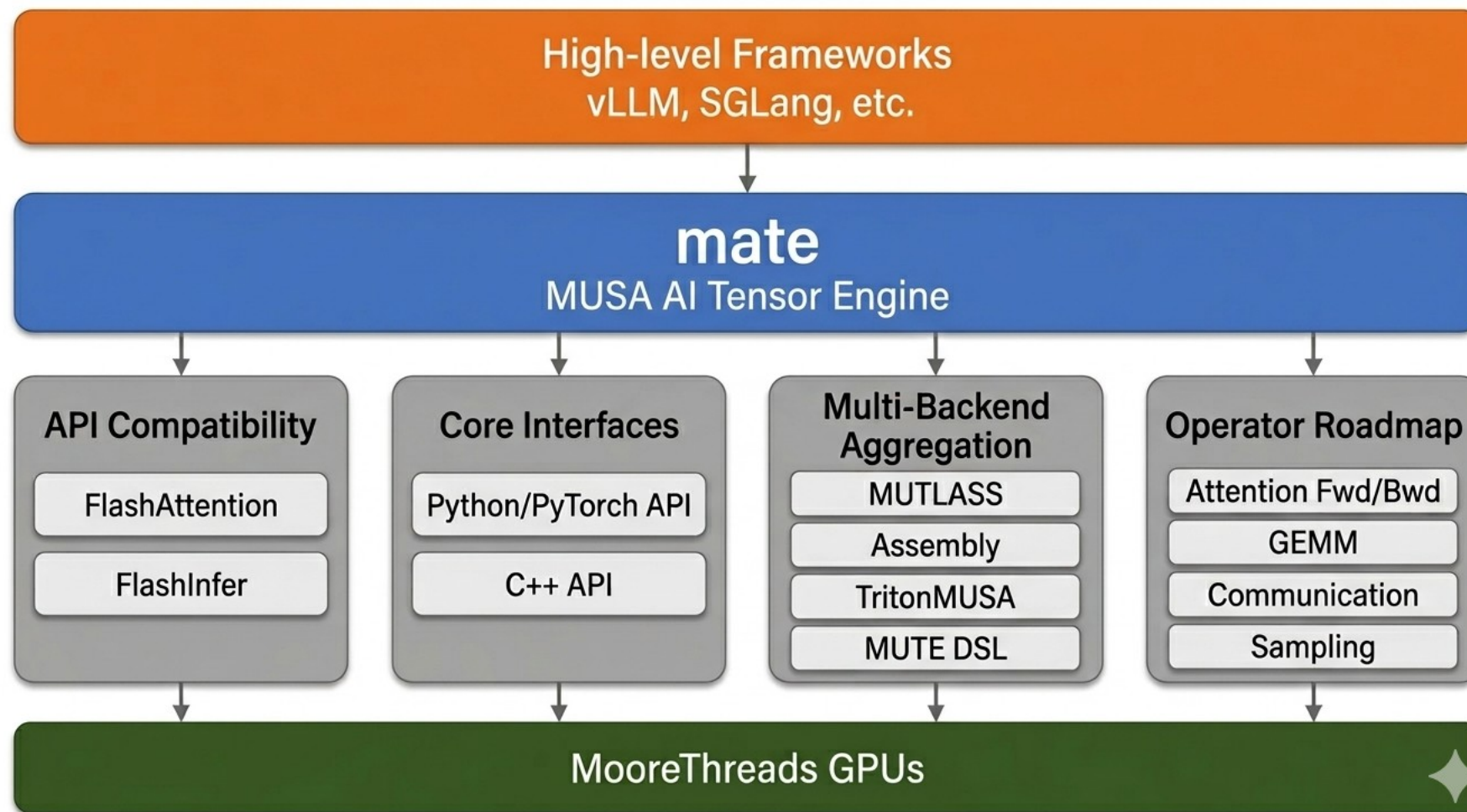
Torchada & mthreads-ml-py (import pymtml as pynvml)



<https://github.com/MooreThreads/torchada>
<https://github.com/MooreThreads/mthreads-ml-py>



MATE



Part 03 Implementation Highlights

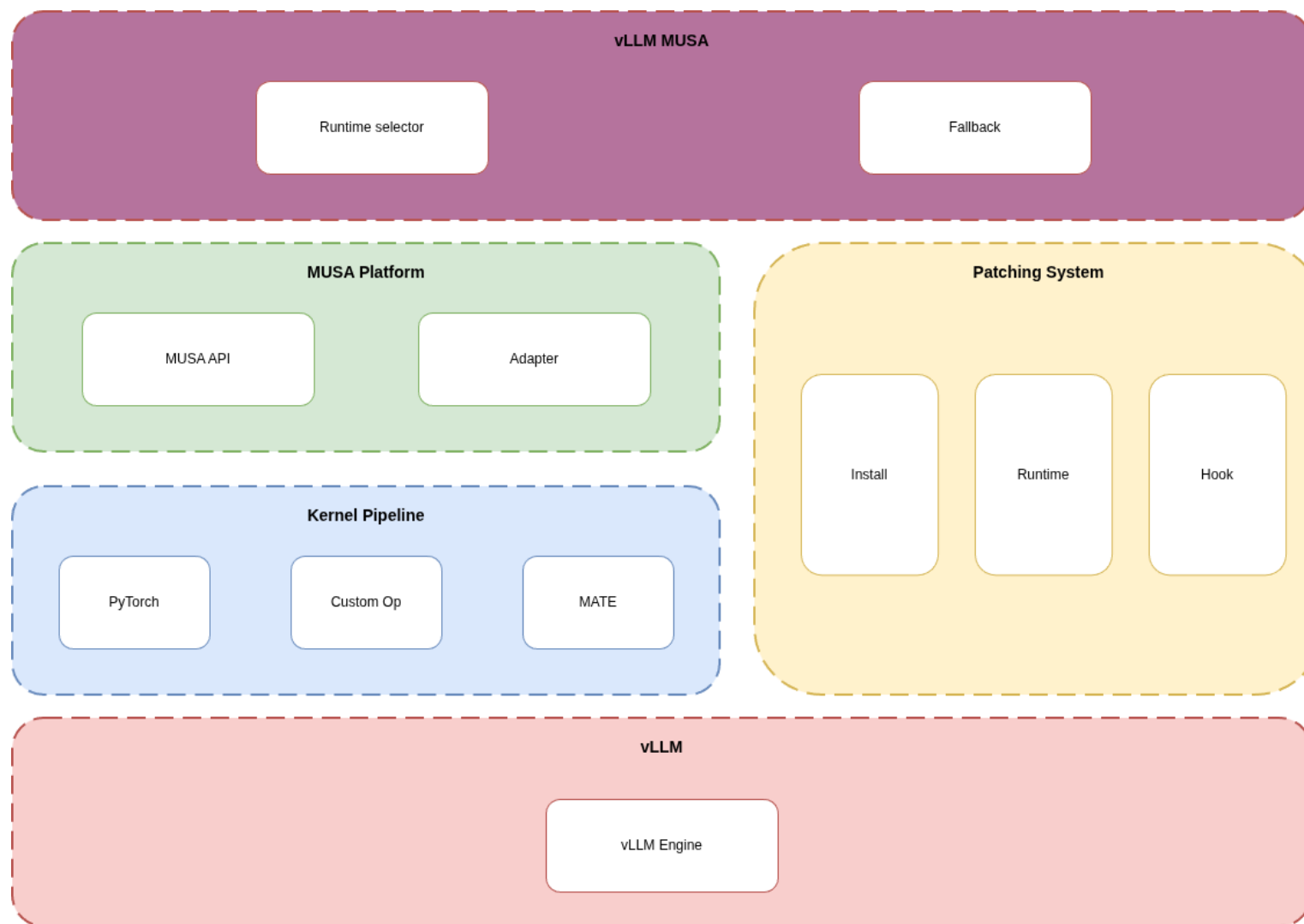


Implementation deep dive — compat

- Platform & Adapter (musa.py parity).
- Small patching system for minimal code replacements.
- Kernel pipeline: Pytorch → Custom op → MATE.
- Runtime selector & fallback.



Implementation deep dive — compat



Platform abstraction — musa.py parity

- musa.py mirrors CUDA API: device query, streams/events, alloc hooks.
- Adapter handles semantic differences (sync/async, capability flags).
- Result: most framework calls run unchanged; test edge cases (streams/events).

```
614 @classmethod
615 @with_nvml_context
616 def get_device_uuid(cls, device_id: int = 0) -> str:
617     physical_device_id = cls.device_id_to_physical_device_id(device_id)
618     handle = pynvml.nvmlDeviceGetHandleByIndex(physical_device_id)
619     return pynvml.nvmlDeviceGetUUID(handle)
620
621 @classmethod
622 @with_nvml_context
623 def get_device_total_memory(cls, device_id: int = 0) -> int:
624     physical_device_id = cls.device_id_to_physical_device_id(device_id)
625     handle = pynvml.nvmlDeviceGetHandleByIndex(physical_device_id)
626     return int(pynvml.nvmlDeviceGetMemoryInfo(handle).total)
627
628 @classmethod
629 @with_nvml_context
630 def is_fully_connected(cls, physical_device_ids: list[int]) -> bool:
631     """
632     query if the set of gpus are fully connected by nvlink (1-hop)
633     """
634     handles = [pynvml.nvmlDeviceGetHandleByIndex(i) for i in physical_device_ids]
635     for i, handle in enumerate(handles):
636         for j, peer_handle in enumerate(handles):
637             if i < j:
638                 try:
639                     p2p_status = pynvml.nvmlDeviceGetP2PStatus(
640                         handle,
641                         peer_handle,
642                         pynvml.NVML_P2P_CAPS_INDEX_NVLINK,
643                     )
```

```
479 @classmethod
480 @with_mtml_context
481 def get_device_uuid(cls, device_id: int = 0) -> str:
482     physical_device_id = cls.device_id_to_physical_device_id(device_id)
483     handle = pynvml.nvmlDeviceGetHandleByIndex(physical_device_id)
484     return pynvml.nvmlDeviceGetUUID(handle)
485
486 @classmethod
487 @with_mtml_context
488 def get_device_total_memory(cls, device_id: int = 0) -> int:
489     physical_device_id = cls.device_id_to_physical_device_id(device_id)
490     handle = pynvml.nvmlDeviceGetHandleByIndex(physical_device_id)
491     return int(pynvml.nvmlDeviceGetMemoryInfo(handle).total)
492
493 @classmethod
494 @with_mtml_context
495 def is_fully_connected(cls, physical_device_ids: list[int]) -> bool:
496     """
497     query if the set of gpus are fully connected by mtlink (1-hop)
498     """
499     handles = [pynvml.nvmlDeviceGetHandleByIndex(i) for i in physical_device_ids]
500     for i, handle in enumerate(handles):
501         for j, peer_handle in enumerate(handles):
502             if i < j:
503                 try:
504                     p2p_status = pynvml.nvmlDeviceGetP2PStatus(
505                         handle,
506                         peer_handle,
507                         pynvml.NVML_P2P_CAPS_INDEX_NVLINK,
508                     )
```

```
>>> import torch
>>> import torchdada
>>> torch.musa.is_available()
True
>>> torch.cuda.Stream()
torch.musa.Stream device=musa:0 musa_stream=0x555a5c
>>> torch.tensor(0, device="cuda")
tensor(0, device='musa:0')
```



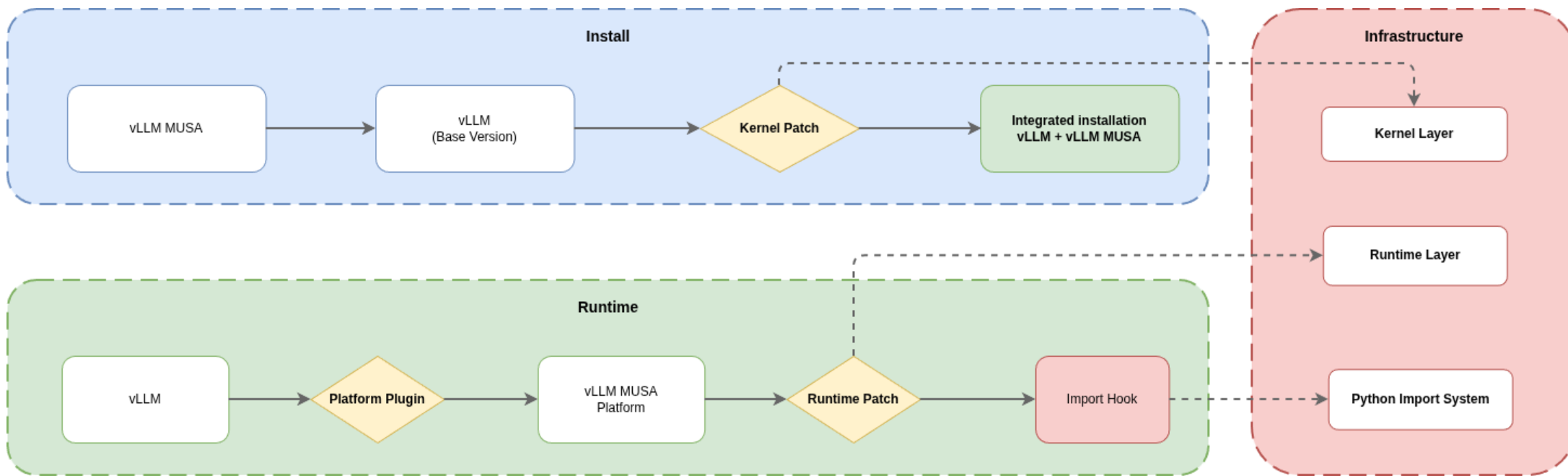
Small patching system — minimal & reversible vLLM



- Rationale: some vLLM internals still assume CUDA.
- Implementation: minor source code patches and registerable patches (monkey-patch / import hooks) applied only in plugin mode.
- Principle: minimal invasion, testable, reversible.



Small patching system — minimal & reversible



Kernel pipeline: validate → optimize

Focus points: KV layout, memory tiling, block sizes → flash attention+fused kernels

Step A Pytorch

```
def forward_native(
    self,
    x: torch.Tensor
) -> torch.Tensor:
    return F.swish_glu(x)
```

Run compatible implementation with torchada to validate correctness and profile hotspots

Step B Custom op

```
from vllm_musa import _ops

def fused_experts_impl(
    hidden_states: torch.Tensor,
    w1: torch.Tensor,
    w2: torch.Tensor,
    topk_weights: torch.Tensor,
    topk_ids: torch.Tensor,
    # ...omitting other parameters...
) -> torch.Tensor:
    """
    ops.musa_fused_gemv_moe(.....)
    """
```

Isolation of vLLM's custom operators, and optimized specifically for the musa to achieve higher efficiency.

Step C MATE

```
import mate.flashmla
flashmla.flash_mla_with_kvcache(
    q,
    k_cache,
    block_table,
    cache_seqlens,
    # ...omitting other parameters...
)
```

Ensure parameter alignment. Dynamic distribution. For production.



Runtime selector & fallback

Auto fallback to compat
implementation when
unsupported.

musa.p
y

Custom

op
ENV, can switch from the default OOT
implementation to the native
(e.g.,
VLLM_MUSA_CUSTOM_OP_USE_NATIVE)

mat
e

Register kernels with dynamic
routing (seq_len/head_size).

Env override for testing
(e.g., VLLM_USE_MATE=1).



Runtime selector & fallback

```
@RMSNorm.register_oop
class MusaRMSNorm(RMSNorm):
    def forward_oop(
        self,
        x: torch.Tensor,
        residual: torch.Tensor | None = None,
    ) -> torch.Tensor | tuple[torch.Tensor, torch.Tensor]:
        if envs.VLLM_MUSA_CUSTOM_OP_USE_NATIVE.get():
            return self.forward_native(x, residual)

        if self.variance_size_override is not None:
            return self.forward_native(x, residual)

        add_residual = residual is not None
        if add_residual:
            return fused_add_rms_norm(
                x, residual, self.weight.data, self.variance_epsilon
            )
        else:
            out = nn.functional.rms_norm(
                x, (self.hidden_size,), self.weight.data, self.variance_epsilon
            )
            return out
```



Risks & mitigations

- Driver/runtime/framework version upgrades → Maintain version matrix and continuous integration (CI).
- Kernel correctness/performance → Unit tests + holistic guarding.

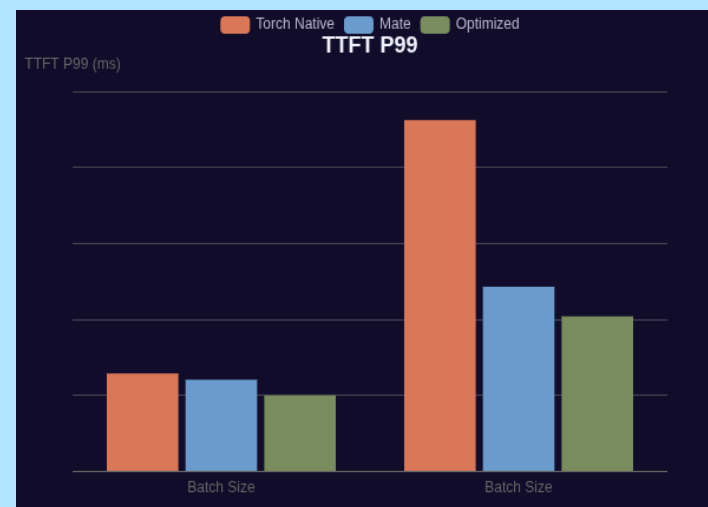
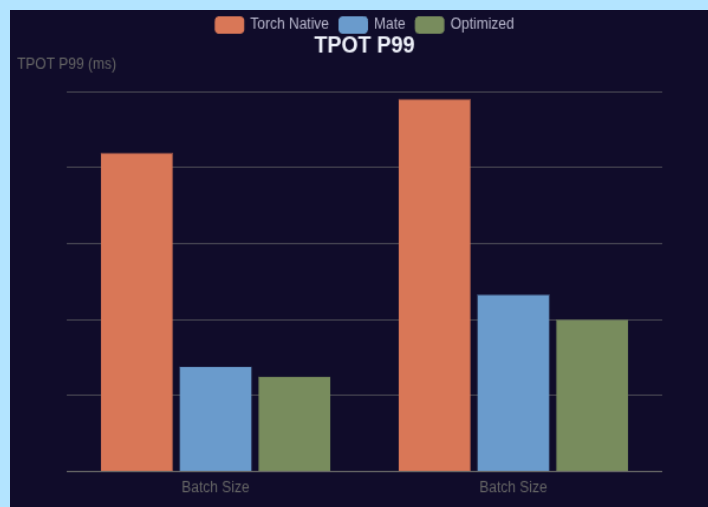


Part 04rf, Ops, Takeaways



Performance — baseline vs mate vs optimization

- Compare groups: Torch native+Triton → Custom op + MATE → continuous optimization
- Metrics: throughput (tokens/s), TTFT P99, TPOT P99.



Operational notes & monitoring

- Use **mtthreads-gmi** for topology & utilization, and cluster monitoring.
- Keep driver/runtime/test matrix in CI; nightly runs for regressions.
- Rollout strategy: canary → small cluster → deployment with monitoring.



Takeaways & next steps

- Practical path: CUDA-compat layer + vendor kernels + small reversible patches.
- Next: expand MATE coverage, upstream stable plugin hooks into vLLM, publish benchmarks.
- Call to action: collaborate on upstream hooks & shared benchmark suites.





摩尔线程公众号



摩尔线程官网



摩尔学院官网

Thanks.

